

A

# DNF 刷图代码逐段讲解版

修：修复乱 去 大 分 保U “上 代 ， 下 ”

R 两件事：，代 使 前工作区 已 常 中  
； 二，去 封 大U 和 制 ， 你可以 从上  
下 ， 不会再出 大 A

前 参与刷图主 件：main.py detector.py  
door\_strategy.py minimap\_astar.py minimap\_nav.py game.py  
olaplug 大，但不 R 主刷图 代 ， 以R 仍  
在 ， 不 主

## 目录

- 1. main.py：入口
- 2. detector.py：YOLO 大
- 3. door\_strategy.py：分
- 4. minimap\_astar.py：A 寻
- 5. minimap\_nav.py：小地图 别与
- 6. game.py：动作 制 取 卡住 复
- 7. minimap\_route\_demo.py：导

## 第一章：main.py

这个文件负责什么：它 个刷图 序 入口 口 口 图  
YOLO 小地图导 ， 再 交 Game 去

### 片段 1：文件头、导入、路径兼容和窗口常量

main.py

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
@File      : main.py
@Desc      : Win32-based DNF capture loop
"""

from __future__ import annotations

import pathlib
import time
from typing import Optional, Tuple

import cv2
import numpy as np
import pyautogui
import win32con
import win32gui
from loguru import logger

from dnf.game import Game
from dnf.minimap_nav import MiniMapNavigator

temp = pathlib.PosixPath
pathlib.PosixPath = pathlib.WindowsPath

from dnf.detector import Detector

WINDOW_OFFSET_X = 10
WINDOW_OFFSET_Y = 10
WINDOW_TITLE_KEYWORD = "地下城与勇士"
WINDOW_CLASS_NAMES = {"地下城与勇士", "地下城与勇士创新世纪"}
```

### 这一段的作用

R A 先主序 库全导 cv2 处图像, numpy 图, pyautogui 图, win32gui 与 win32con Windows 口作, Game 动作层, MiniMapNavigator 导层, Detector YOLO 层

### 执行逻辑

R 个 兼容小 巧: pathlib.PosixPath 临 WindowsPath, 为了 免 些依 在 Windows 下 出 下 几个



```

logger.info(
    "绑定窗口: hwnd={}, title={}, class={}",
    hwnd,
    candidates[0][2],
    candidates[0][3],
)
return hwnd

```

A

### 这一段的作用

R 代 在 可 口中, DNF 口 出 , 并 回它 句  
hwnd 5R 个句 可以 为 口在A 5 份

### 执行逻辑

序先 历全 口, 再 序做 : 不可 不 , 和 名 为 不 ,  
和 名 不匹 DNF 关 F 也不 只 像 DNF 口 会 取大小,  
并 加入候 列 后从候 大 个 口作为

### 它是干什么的

它 上不 后台 定, 定 " 住哪 个 口" R , 因为后  
图 以R 个 口为中

## 片段 3: 把窗口恢复并摆到固定位置

函 `_place_window()`

```

def _place_window(hwnd: int) -> None:
    win32gui.ShowWindow(hwnd, win32con.SW_RESTORE)
    left, top, right, bottom = win32gui.GetWindowRect(hwnd)
    width = max(1, right - left)
    height = max(1, bottom - top)
    win32gui.SetWindowPos(
        hwnd,
        win32con.HWND_TOPMOST,
        WINDOW_OFFSET_X,
        WINDOW_OFFSET_Y,
        width,
        height,
        win32con.SWP_SHOWWINDOW,
    )

```

这一段的作用

到 口后， 序会 它 复出 并 动到左上 固定位 ， R 图 定

执行逻辑

SW\_RESTORE 口从 小化 复 后 取 口 前大小，再  
SetWindowPos() 它 到 偏 位 ， 并

它是干什么的

动化 口位 乱变， R 就 为了 后 图区域 定下

片段 4：计算客户区并截图

函 \_get\_client\_region() 与 \_capture\_client()

```
def _get_client_region(hwnd: int) -> Tuple[int, int, int, int]:
    left_top = win32gui.ClientToScreen(hwnd, (0, 0))
    client_rect = win32gui.GetClientRect(hwnd)
    left = int(left_top[0])
    top = int(left_top[1])
    width = int(client_rect[2] - client_rect[0])
    height = int(client_rect[3] - client_rect[1])
    if width <= 0 or height <= 0:
        raise RuntimeError("获取游戏客户区失败")
    return left, top, width, height

def _capture_client(hwnd: int) -> Tuple[np.ndarray, Tuple[int, int]]:
    left, top, width, height = _get_client_region(hwnd)
    img = pyautogui.screenshot(region=(left, top, width, height))
    img_np = np.array(img)
    return img_np, (width, height)
```

这一段的作用

R 代 下 R 客 区，也就 内容  
区域，不包 和

执行逻辑

序先 `ClientToScreen()` 到客 区左上 屏幕坐 , 再  
`GetClientRect()` 到客 区宽 之后 R 参  
`pyautogui.screenshot()` 图, 并 `numpy` , 供 OpenCV 和 YOLO 使

它是干什么的

R 就 个 别 原始 入 图, 后 别和导 从V

片段 5: 主函数初始化

函 `main()` 头

```
def main() -> None:
    device_type = ""
    detector = Detector(device_type)
    navigator = MiniMapNavigator("generic")

    hwnd = _find_dnf_window()
    _place_window(hwnd)
```

这一段的作用

主函 始 , 会先 大 器和小地图导 器 准备好, 后 并 好 DNF  
口

执行逻辑

`device_type = ""` 代 器 已决定 GPU CPU  
`MiniMapNavigator("generic")` 前 `generic` 地图 处 小地  
图

它是干什么的

R 入主 前 准备 型和导 器只 初始化 , 不 帧

片段 6: 主循环里截图、检测和导航

函 main() 中

```
while True:
    start_time = time.time()
    try:
        img_np, (width, height) = _capture_client(hwnd)
    except Exception as exc:
        logger.exception("截图失败: {}", exc)
        time.sleep(0.2)
        continue

    img, obj = detector.detect(img_np)
    logger.info("obj: {}", obj)

    frame_bgr = cv2.cvtColor(img_np, cv2.COLOR_RGB2BGR)
    route_snapshot = navigator.build_route_snapshot(frame_bgr, obj or [])
    route_direction = (
        route_snapshot.next_room_direction.upper()
        if route_snapshot.next_room_direction
        else None
    )
```

### 这一段的作用

R 主 前半 : 先 图, 再做大 , 再做小地图导 , 到 向

### 执行逻辑

先 图; 图失 , 则 常 作 后 图 功后, 图 YOLO 器, 到 和可 化图 后 原图 BGR, S 导 器 导 器 会 小地图 别 和大 家信 合 , 到 前 和下 向

### 它是干什么的

R 于 " 图并 " , 它只 出判 , 不 实

## 片段 7: 把方向交给动作层执行

函 main() 尾

```

        logger.info(
            "route: current={}, boss={}, query={}, elite={}, down={},
direction={}, door={}, scores={}",
            route_snapshot.current_room,
            route_snapshot.boss_room,
            route_snapshot.query_room,
            route_snapshot.elite_room,
            route_snapshot.down_room,
            route_direction,
            route_snapshot.selected_door_center,
            route_snapshot.debug_scores,
        )

    game = Game(obj, width, height, route_direction)
    game.run()

    display = cv2.resize(img, (640, 360))
    cv2.imshow("112233", display)
    logger.info("处理时间: {}", time.time() - start_time)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

    cv2.destroyAllWindows()

A
if __name__ == "__main__":
    main()

```

### 这一段的作用

R 别和导 交 动作层 `Game`， 动作层决定R 帧 东 动

### 执行逻辑

先 出 偏，再创 `Game` 对，并 尺寸和 向交 它 `game.run()` 会 帧 为 后 会 个 后 口，便 发 别

### 它是干什么的

`main.py` 在R 完 了 己 使命：“ ” 和 “ ”，交 “动” 分去



## 第二章：detector.py

这个文件负责什么：YOLO 型 别大 中 家 Boss 币和 , 序 只

### 片段 1：初始化检测器

Detector    \_\_init\_\_

```
class Detector:
    def __init__(self, device_type: str = ""):
        self.img_size = 640
        self.conf_thres = 0.75
        self.iou_thres = 0.45
        self.hide_labels = False
        self.hide_conf = False

        self.device = device_type or ("0" if torch.cuda.is_available() else "cpu")

        self.weights = str(Path(__file__).resolve().parent / "shzn.pt")
        self.model = YOLO(self.weights)
        self.names = self.model.names
        self.colors = [[random.randint(0, 255) for _ in range(3)] for _ in range(len(self.names))]
```

#### 这一段的作用

R 代 YOLO 型 加 , 并 参

#### 执行逻辑

img\_size 入 型 尺寸, conf\_thres 信度 值, iou\_thres NMS 去 值 序会 动优先使 GPU, 不 再 回 CPU 型 件 地 shzn.pt

#### 它是干什么的

R 器 R , 序后 就不 哪些 哪些

片段 2：标准化标签和坐标格式

两个 工具函

```
@staticmethod
def _normalize_label(label: str) -> str:
    label = str(label).strip().lower()
    if "door" in label:
        return "door"
    return label

@staticmethod
def _xyxy_to_xywh_top_left(xyxy: np.ndarray) -> List[float]:
    x1, y1, x2, y2 = [float(v) for v in xyxy]
    return [x1, y1, max(0.0, x2 - x1), max(0.0, y2 - y1)]
```

这一段的作用

型吐出 小写字 串, 关 叫  
door ; 同 坐 从 xyxy xywh

执行逻辑

R 做后, 后 动作层和导 层就不 各 复 坐 和 了

它是干什么的

R 个典型 层, 后 块使 同 入

片段 3：检测函数入口

函 detect() 头

```
def detect(self, img0: np.ndarray) -> Tuple[np.ndarray | None, list | None]:
    if img0 is None:
        return None, None

    img_bgr = cv2.cvtColor(img0, cv2.COLOR_RGB2BGR)
    result = self.model.predict(
        source=img_bgr,
        imgsz=self.img_size,
        conf=self.conf_thres,
        iou=self.iou_thres,
```

```
device=self.device,  
verbose=False,  
)[0]
```

### 这一段的作用

R 代 始对 前 图做 YOLO

### 执行逻辑

如 入图 为 , 就 回 , 序出 否则先 RGB BGR,  
再 图像 型

### 它是干什么的

从R 始, 序就 " " 了

## 片段 4: 整理检测结果

函 detect() 后半

```
annotator = Annotator(img_bgr.copy(), line_width=3, example=self.names)  
obj = []  
  
if result.bboxes is not None and len(result.bboxes) > 0:  
    boxes_xyxy = result.bboxes.xyxy.cpu().numpy()  
    boxes_conf = result.bboxes.conf.cpu().numpy()  
    boxes_cls = result.bboxes.cls.cpu().numpy().astype(int)  
  
    for xyxy, conf, cls in zip(boxes_xyxy, boxes_conf, boxes_cls):  
        raw_label = self.names[int(cls)]  
        label = self._normalize_label(raw_label)  
        xywh = self._xyxy_to_xywh_top_left(xyxy)  
        conf = round(float(conf), 2)  
  
        annotator.box_label(  
            xyxy,  
            raw_label if self.hide_conf else f"{raw_label} {conf:.2f}",  
            color=tuple(self.colors[int(cls)]),  
        )  
        obj.append({label: {"xywh": xywh, "conf": conf}})  
  
return annotator.result(), obj
```

这一段的作用

型原始 出 序 容 使 对 列 , 同 到图上

执行逻辑

个 会 别 信度和位 别 准化, 位 坐 , 后  
存入 obj 列 R 个 obj 就 后 东 依

它是干什么的

前 序 化 , 后 人 可 化图

A

A

第三章: door\_strategy.py

这个文件负责什么: A 告 序 “下 在右 ” , R 在 前 多  
出 像 “右 ”

片段 1: 门数据结构和方向取反

件

```
@dataclass
class DoorCandidate:
    bbox: Sequence[float]
    center: Point

def opposite_direction(direction: Optional[str]) -> Optional[str]:
    mapping = {
        "left": "right",
        "right": "left",
        "up": "down",
        "down": "up",
    }
    return mapping.get(direction)
```

这一段的作用

先 封 , 再 供 个 “ 反 向 ” 工具函

执行逻辑

后 中 , 也 上 向 不 反向 头, 以R 两  
个工具 基

它是干什么的

R 底层 准备层

片段 2：计算偏移和方向匹配

两个小函

```
def _axis_offsets(player: Point, door: Point) -> Tuple[float, float]:
    dx = door[0] - player[0]
    dy = door[1] - player[1]
    return dx, dy

def _direction_match(expected_direction: str, dx: float, dy: float, margin:
float) -> bool:
    if expected_direction == "right":
        return dx > margin
    if expected_direction == "left":
        return dx < -margin
    if expected_direction == "up":
        return dy < -margin
    if expected_direction == "down":
        return dy > margin
    return True
```

这一段的作用

判 对 家到底偏在哪 , 以及它 不 合 前 向

执行逻辑

margin 容差 如 向右 , 在 家右 , 不 只 刚好 中

它是干什么的

R A 向和屏幕中 位

### 片段 3: 核心选门算法

函 `choose_best_door()`

```
def choose_best_door(
    doors: Sequence[DoorCandidate],
    player_center: Point,
    expected_direction: Optional[str],
    last_direction: Optional[str] = None,
    margin: float = 16.0,
) -> Optional[DoorCandidate]:
    if not doors:
        return None
    if expected_direction is None:
        return min(doors, key=lambda item: abs(item.center[0] -
player_center[0]) + abs(item.center[1] - player_center[1]))

    reverse_direction = opposite_direction(last_direction)
    candidates = []
    fallback = []

    for door in doors:
        dx, dy = _axis_offsets(player_center, door.center)
        abs_dx = abs(dx)
        abs_dy = abs(dy)
        match = _direction_match(expected_direction, dx, dy, margin)

        if expected_direction in ("left", "right"):
            forward = dx if expected_direction == "right" else -dx
            sideways = abs_dy
        else:
            forward = dy if expected_direction == "down" else -dy
            sideways = abs_dx

        reverse_penalty = 0.0
        if reverse_direction and expected_direction == reverse_direction:
            reverse_penalty = 2000.0

        score = (-forward + reverse_penalty, sideways, abs_dx + abs_dy)
        if match:
            candidates.append((score, door))
        fallback.append((score, door))

    if candidates:
```

```

    candidates.sort(key=lambda item: item[0])
    return candidates[0][1]
```

```

    fallback.sort(key=lambda item: item[0])
    return fallback[0][1]
```

这一段的作用

R 它会对 分,再 出 合 前 向

执行逻辑

回 ; 向 , 向 , 对 三  
个 度: 前 侧向偏 , 同 会 反 向 加 个 大 S  
匹 向 会 入优先候 ; 全 会 入 底候 优先从匹 候  
中取分 好 , 底

它是干什么的

它 不 复 学, 个 实 启发 分器: 尽 合 向 又不  
偏 又不

第四章: minimap\_astar.py

这个文件负责什么: 做 A 寻 它 出 “下 哪 ”, 不  
内 像

片段 1: 节点结构与曼哈顿启发

件

```

@dataclass(order=True)
class _Node:
    f: int
    position: GridPoint
    g: int = 0
    h: int = 0
    parent: Optional["_Node"] = None
```

```
def heuristic(a: GridPoint, b: GridPoint) -> int:
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

A

### 这一段的作用

定义 A 和启发函 `f = g + h` A 分

### 执行逻辑

`g` 已 代价, `h` 到 估 代价, R 哈 , 因为地图只  
上下左右 动

### 它是干什么的

R 个寻 地基

## 片段 2：邻居扩展顺序

函 `_neighbors()`

```
def _neighbors(position: GridPoint, grid: Sequence[Sequence[int]], priority:
str):
    x, y = position
    directions = {
        "right": ((0, 1), (1, 0), (-1, 0), (0, -1)),
        "left": ((0, -1), (-1, 0), (1, 0), (0, 1)),
        "up": ((-1, 0), (0, -1), (0, 1), (1, 0)),
        "down": ((1, 0), (0, 1), (0, -1), (-1, 0)),
        "default": ((0, 1), (1, 0), (-1, 0), (0, -1)),
    }
```

### 这一段的作用

决定 前 应 优先 哪个 向 展 居 如优先 右, 就先 右再 别  
向

### 执行逻辑

R 不会 变 , 但在多 , 会 响 序 偏向哪

### 它是干什么的

你可以 它 “同分 况下 偏好”



### 片段 3: A 星主循环

函 `a_star()`

```
def a_star(grid, start, end, priority="right"):
    if start == end:
        return [start]

    open_list = []
    start_node = _Node(f=0, position=start, g=0, h=heuristic(start, end),
parent=None)
    start_node.f = start_node.g + start_node.h
    heapq.heappush(open_list, start_node)
    closed = set()
    best_g = {start: 0}

    while open_list:
        current = heapq.heappop(open_list)
        if current.position in closed:
            continue
        closed.add(current.position)

        if current.position == end:
            path = []
            node = current
            while node is not None:
                path.append(node.position)
                node = node.parent
            return list(reversed(path))

        for next_pos in _neighbors(current.position, grid, priority):
            next_g = current.g + 1
            if next_g >= best_g.get(next_pos, 10 ** 9):
                continue
            best_g[next_pos] = next_g
            next_h = heuristic(next_pos, end)
            heapq.heappush(
                open_list,
                _Node(
                    f=next_g + next_h,
                    position=next_pos,
                    g=next_g,
                    h=next_h,
                    parent=current,
```

```
    ),  
    )  
    return None
```

### 这一段的作用

R 就 完 A : 不 取出 前 优 , 到 到 并回 出

### 执行逻辑

A  
列 “可 值 ” 取出 f 小 个, 展  
居; 如 到了 , 就 parent 回

### 它是干什么的

它在 套 序 , 导 层

## 片段 4: 只取路径第一步方向

函 `judge_direction()` 与 `next_direction()`

```
def judge_direction(starting_point: GridPoint, end_point: GridPoint) ->  
Optional[str]:  
    x1, y1 = starting_point  
    x2, y2 = end_point  
    if x2 > x1:  
        return "down"  
    if x2 < x1:  
        return "up"  
    if y2 > y1:  
        return "right"  
    if y2 < y1:  
        return "left"  
    return None  
  
def next_direction(grid, start, end, priority="right") -> Optional[str]:  
    path = a_star(grid, start, end, priority=priority)  
    if not path or len(path) < 2:  
        return None  
    return judge_direction(path[0], path[1])
```

这一段的作用

压 个 信 : 下 哪

执行逻辑

前 并不 交 动作层, 只取前两 之 向, R 动作  
层只 关 “下 在上 下 左 右哪 ”

它是干什么的

R A 和动作层 出口 A 不 制 动, 只 出 向

第五章: minimap\_nav.py

这个文件负责什么: 它 小地图导 中 : 小地图 匹 图 别 前 和  
A , 并尝 向关 到 前 中

片段 1: 地图规格与路线快照结构

```
@dataclass
class MapSpec:
    name: str
    crop_rect_1067: Rect
    minimap_width: int
    minimap_height: int
    rows: int
    cols: int
    room_grid: List[List[int]]

@dataclass
class RouteSnapshot:
    current_room: Optional[Point]
    boss_room: Optional[Point]
    query_room: Optional[Point]
    elite_room: Optional[Point]
    down_room: Optional[Point]
    next_room_direction: Optional[str]
```

```
selected_door_center: Optional[Tuple[float, float]]
debug_scores: Optional[Dict[str, float]] = None
```

### 这一段的作用

MapSpec 地图 么 , RouteSnapshot 导 器 出 完

### 执行逻辑

前 像 , 后 像 帧 出 包

### 它是干什么的

R 导 层

## 片段 2：地图字典

MAP\_SPECS

```
def _all_walkable(rows: int, cols: int) -> List[List[int]]:
    return [[0 for _ in range(cols)] for _ in range(rows)]
```

```
MAP_SPECS = {
    "generic": MapSpec(
        name="generic",
        crop_rect_1067=(893, 52, 1055, 142),
        minimap_width=162,
        minimap_height=90,
        rows=5,
        cols=9,
        room_grid=_all_walkable(5, 9),
    ),
    "haibolun": MapSpec(
        name="haibolun",
        crop_rect_1067=(966, 52, 1056, 124),
        minimap_width=90,
        minimap_height=72,
        rows=4,
        cols=5,
        room_grid=_all_walkable(4, 5),
    ),
}
```

```
),  
}
```

### 这一段的作用

定义不同地图 剪区域和

### 执行逻辑

前R 份代 中 `room_grid` 基 全 0, 也就 位 可

### 它是干什么的

它告 导 器 "R 地图应 么切 么分 子 A 在哪 上 "

## 片段 3：初始化导航器并加载模板

`MiniMapNavigator` `__init__`

```
def __init__(self, map_name: str = "generic", assets_dir: Optional[Path] =  
None, threshold: float = 0.7):  
    if map_name not in MAP_SPECS:  
        map_name = "generic"  
    self.map_name = map_name  
    self.spec = MAP_SPECS[map_name]  
    self.threshold = threshold  
    self.assets_dir = assets_dir or (Path(__file__).resolve().parent / "res")  
    self.templates = {  
        "hero": self._load_template("map_hero.png"),  
        "boss": self._load_template("map_boss.png"),  
        "query": self._load_template("map_query.png"),  
        "query1": self._load_template("map_query1.png"),  
        "query3": self._load_template("map_query3.png"),  
        "query4": self._load_template("map_query4.png"),  
        "query5": self._load_template("map_query5.png"),  
        "elite": self._load_template("map_elite.png"),  
        "special": self._load_template("map_special_room.png"),  
    }  
    down_template = self._load_template_optional("map_down.png")  
    if down_template is not None:  
        self.templates["down"] = down_template  
    self.last_direction: Optional[str] = None
```

### 这一段的作用

小地图导 器准备好, 并 图 入内存

执行逻辑

先 定 前地图 , 再加 图 Boss 图 号 图 图  
last\_direction 住上 向, 后 会 免 头

它是干什么的

它 于 导 器 上 “ 别字典”

片段 4：换算小地图裁剪区域

函 \_scaled\_crop\_rect() 与 extract\_minimap()

```
def _scaled_crop_rect(self, frame: np.ndarray) -> Rect:
    frame_h, frame_w = frame.shape[:2]
    x1, y1, x2, y2 = self.spec.crop_rect_1067
    scale_x = frame_w / 1067.0
    scale_y = frame_h / 600.0
    return (
        int(round(x1 * scale_x)),
        int(round(y1 * scale_y)),
        int(round(x2 * scale_x)),
        int(round(y2 * scale_y)),
    )

def extract_minimap(self, frame: np.ndarray) -> np.ndarray:
    x1, y1, x2, y2 = self._scaled_crop_rect(frame)
    return frame[y1:y2, x1:x2]
```

A A  
这一段的作用  
图中 小地图 出 , 同 对不同分 做 例

执行逻辑

地图 剪 基准分 写 , R 会 前 图尺寸  
, 保 不同 口大小下 到 小地图区域

它是干什么的

R 小地图 别 , 图, 匹 全 会偏

## 片段 5：模板匹配核心

函 `_match_template()` 与 `_match_template_debug()`

```
def _match_template(self, image: np.ndarray, template: np.ndarray, threshold:
Optional[float] = None):
    threshold = self.threshold if threshold is None else threshold
    image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    template_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
    result = cv2.matchTemplate(image_gray, template_gray,
cv2.TM_CCOEFF_NORMED)
    ys, xs = np.where(result >= threshold)
    matches = []
    h, w = template_gray.shape[:2]
    for y, x in zip(ys, xs):
        matches.append((x + w / 2.0, y + h / 2.0))
    return matches
```

A

### 这一段的作用

R 小地图 别 : 图, 在小地图上 动 , 哪 像它

### 执行逻辑

先 度, 再 `cv2.matchTemplate()` 做 关 匹 , 后 值 位  
中

### 它是干什么的

R 导 器 “ 别 ” , 专 小地图上 图

## 片段 6：像素位置换成房间坐标

函 `compute_room_id()`

```
def compute_room_id(self, x: float, y: float, minimap_image: np.ndarray) ->
Point:
    height, width = minimap_image.shape[:2]
    cell_w = width / self.spec.cols
    cell_h = height / self.spec.rows
    row = int(y // cell_h)
    col = int(x // cell_w)
    row = max(0, min(self.spec.rows - 1, row))
```

```
col = max(0, min(self.spec.cols - 1, col))
return row, col
```

### 这一段的作用

图中像坐坐

### 执行逻辑

小地图宽和列，先个子尺寸，再出图在哪哪列

### 它是干什么的

A 不吃像坐，只吃坐，以做R

## 片段 7：识别当前房间、Boss 房、问号房等

函 `detect_room_markers()`

```
def detect_room_markers(self, frame: np.ndarray) -> Dict[str,
Optional[Point]]:
    minimap = self.extract_minimap(frame)
    current_room = None
    boss_room = None
    query_room = None
    elite_room = None
    down_room = None

    hero_matches = self._match_template(minimap, self.templates["hero"])
    if hero_matches:
        current_room = self.compute_room_id(*hero_matches[0], minimap)

    boss_matches = self._match_template(minimap, self.templates["boss"])
    if boss_matches:
        boss_room = self.compute_room_id(*boss_matches[0], minimap)

    query_matches = self._match_template(minimap, self.templates["query"])
    if not query_matches:
        query_matches = self._match_template(minimap,
self.templates["query1"])
    if not query_matches:
        query_matches = self._match_template(minimap,
self.templates["query3"])
```



```
if not query_matches:
    query_matches = self._match_template(minimap,
self.templates["query4"])
if not query_matches:
    query_matches = self._match_template(minimap,
self.templates["query5"])
if query_matches:
    query_room = self.compute_room_id(*query_matches[0], minimap)
```

### 这一段的作用

小地图上

别出

### 执行逻辑

图 对应 前 , Boss 图 对应 Boss 号 , 代 准备了多个不同 , 匹 到任 个就 到

### 它是干什么的

它在告 序: 在在哪, 值 去 又在哪

## 片段 8: 选择目标房间

函 `_pick_target_room()`

```
def _pick_target_room(
    self,
    current_room,
    query_room,
    down_room,
    elite_room,
    boss_room,
    prefer_special_room: bool,
) -> Optional[Point]:
    candidates = []
    if prefer_special_room:
        candidates.extend([query_room, down_room, elite_room, boss_room])
    else:
        candidates.append(boss_room)

    for room in candidates:
        if room is None:
            continue
```

```
    if current_room is not None and room == current_room:
        continue
    return room
return None
```

### 这一段的作用

从多个候 中 出 前 值 前

### 执行逻辑

前 会优先 号 下层 , 后 Boss 到 个 且  
不 前 , 就 作为

### 它是干什么的

R 在决定 “A 应 从哪 到哪”

## 片段 9：生成完整路线快照

函 `build_route_snapshot()`

```
def build_route_snapshot(self, frame_bgr, detection_objects,
prefer_special_room=True) -> RouteSnapshot:
    room_info = self.detect_room_markers(frame_bgr)
    current_room = room_info["current_room"]
    boss_room = room_info["boss_room"]
    query_room = room_info["query_room"]
    elite_room = room_info["elite_room"]
    down_room = room_info["down_room"]

    target_room = self._pick_target_room(
        current_room=current_room,
        query_room=query_room,
        down_room=down_room,
        elite_room=elite_room,
        boss_room=boss_room,
        prefer_special_room=prefer_special_room,
    )

    next_room_direction = None
    if current_room is not None and target_room is not None:
```

```
next_room_direction = next_direction(deepcopy(self.spec.room_grid),
current_room, target_room, priority="right")
```

### 这一段的作用

R 代 小地图 别 和 A , 出 “下 哪 ”

### 执行逻辑

先 别 , 再 , 再 前 和 丢 `next_direction()` , 从  
到上 下 左 右之

### 它是干什么的

R 小地图 信 变 向 关 合

## 片段 10: 把方向继续关联到门

函 `build_route_snapshot()` 后半

```
selected_door_center = None
if next_room_direction:
    player_center = None
    for item in detection_objects:
        if "player" in item:
            x, y, w, h = item["player"]["xywh"]
            player_center = (x + w / 2.0, y + h / 2.0)
            break
    if player_center is not None:
        selected_door = choose_best_door(
            self._door_candidates_from_objects(detection_objects),
            player_center=player_center,
            expected_direction=next_room_direction,
            last_direction=self.last_direction,
        )
    if selected_door is not None:
        A
        selected_door_center = selected_door.center
        self.last_direction = next_room_direction
```

### 这一段的作用

在已 向以后, 尝 在 前 出 具体

### 执行逻辑

序先 家中 , 再 候 列 , 后 前 分  
, 出 像 向

它是干什么的

R A 不 孤 存在 它会 响 前

第六章: game.py

这个文件负责什么: R 刷图动作 制中 决定R 帧 取 ,  
卡住 复

片段 1: 类级状态与初始化阈值

Game 头

```
class Game:
    _action_cache: Optional[str] = None
    _player: Optional[BoxDict] = None
    _pre_player: Optional[BoxDict] = None
    _index = 0
    _player_missing_count = 0
    _position_history: List[Tuple[float, float]] = []
    _stuck_count = 0
    _last_recover_time = 0.0
    _recover_step = 0
    _last_player_center: Optional[Tuple[float, float]] = None
    _room_entry_position: Optional[Tuple[float, float]] = None
    _room_entry_time = 0.0
    _flow_direction: Optional[str] = None
    _missing_player_recover_threshold = 10
    _route_search_direction: Optional[str] = None
    _route_search_phase = 0
    _route_search_start: Optional[Tuple[float, float]] = None

    def __init__(self, obj: Sequence[dict], width: int, height: int,
direction):
        self._obj = obj or []
        self._width = width
        self._height = height
```

```

self._player_xywh: Optional[List[float]] = None
self._attack_x = 100
self._attack_y = 100
A self._move_x = 20
self._move_y = 20
self._direction = direction

```

A

### 这一段的作用

R 定义了动作 帧 , 以及 击 围 取 围R 些 值

### 执行逻辑

像上 帧动作 几帧位 卡住 入 保存在 变 , 因为它们  
在多帧之 存在

### 它是干什么的

R 动作 “ 仓库” 动化不 帧就 , 以 定 帧

## 片段 2：读取方向提示和检测对象

三个基 工具函

```

def _current_door_direction(self) -> Optional[str]:
    if isinstance(self._direction, (list, tuple)):
        if not self._direction:
            return None
        if Game._index >= len(self._direction):
            return str(self._direction[-1]).upper()
        return str(self._direction[Game._index]).upper()
    if isinstance(self._direction, str) and self._direction.strip():
        return self._direction.strip().upper()
    return None

def _get_cls(self, cls_name: str) -> Optional[BoxDict]:
    for item in self._obj:
        if cls_name in item:
            return item[cls_name]
    return None

def _get_clss(self, cls_name: str) -> List[BoxDict]:
    result = []
    for item in self._obj:

```

```
if cls_name in item:
    A
    result.append(item[cls_name])
return result
```

这一段的作用

取 前 向, 以及从 取出

执行逻辑

`_get_cls()` 取 个, `_get_clss()` 取 全 对 向可以 单字 串, 也可  
以 多 向列

它是干什么的

它们 后 动作判 常 口

片段 3：释放按键和距离函数

基 动作工具

```
def _release_direction_keys(self) -> None:
    for key in ("up", "down", "left", "right"):
        pydirectinput.keyUp(key)

def _release_cached_action(self) -> None:
    if not Game._action_cache:
        self._release_direction_keys()
        return
    for action in Game._action_cache.strip().split("_"):
        pydirectinput.keyUp(action.lower())
    self._release_direction_keys()
    Game._action_cache = None

def _distance(self, a, b) -> float:
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5
```

这一段的作用

基 但 关 工具: 向 存动作 两

执行逻辑

动化 住, 以 变动作前 可

它是干什么的

R 动作 制层 安全基

片段 4: 推 入房方向与更新入口点

入口保

```
def _infer_flow_direction_from_spawn(self, player_center):
    x, y = player_center
    edge_scores = {
        "LEFT": x,
        "RIGHT": self._width - x,
        "UP": y,
        "DOWN": self._height - y,
    }
    nearest_edge = min(edge_scores, key=edge_scores.get)
    opposite = {
        "LEFT": "RIGHT",
        "RIGHT": "LEFT",
        "UP": "DOWN",
        "DOWN": "UP",
    }
    if edge_scores[nearest_edge] > min(self._width, self._height) * 0.28:
        return None
    return opposite[nearest_edge]
```

这一段的作用

刚 哪个 , 他 从哪个 , 并反 出 向

执行逻辑

如 左 , 就 为 从左 , 下 常应 右 值 0.28  
为了 乱

它是干什么的

R 后 “ 刚 就回头” R 件事 依

片段 5: 过滤回头门

## 入口保 函

```
def _filter_backtrack_doors(self, doors, direction_hint):
    if not doors:
        return []
    if not Game._room_entry_position:
        return list(doors)
    now = time.time()
    protect_entry = now - Game._room_entry_time <= 3.0
    if not protect_entry:
        return list(doors)
    ...
    return filtered
```

### 这一段的作用

后 前几 内, 太像 “回头 ”

### 执行逻辑

如 入口太 , 且它又位于 回头 向, 序就会 它 R 可以  
减少 回 动

### 它是干什么的

R 刷图 定 关 保 之

## 片段 6：短按键、轻点方向和方向取反

### 动作底层函

```
def _key_press(self, key: str) -> None:
    pydirectinput.keyDown(key)
    time.sleep(0.08)
    pydirectinput.keyUp(key)
    time.sleep(0.05)

def _tap_direction(self, direction: str, duration: float = 0.12) -> None:
    for action in direction.strip().split("_"):
        pydirectinput.keyDown(action.lower())
    time.sleep(duration)
    for action in direction.strip().split("_"):
```



```
pydirectinput.keyUp(action.lower())
time.sleep(0.04)
```

### 这一段的作用

封基 入动作： 个 ， 个 向

### 执行逻辑

击和 取 ， 卡住 复和 动则 常 向

### 它是干什么的

R 动作层 层 图变 实 事件 底层 器

## 片段 7：选最近目标

函 `_get_nearest()`

```
def _get_nearest(self, cls_objs, direct=None):
    nearest = None
    min_distance = float("inf")
    ...
    distance = (dx ** 2 + dy ** 2) ** 0.5
    if distance < min_distance:
        min_distance = distance
        nearest = {
            "xywh": [obj_x, obj_y, xywh[2], xywh[3]],
            "conf": item.get("conf", 0),
        }
    return nearest
```

### 这一段的作用

在 堆同 ， 如 堆 堆 堆 币中， 家 个

### 执行逻辑

序会先 个 中 ， 再 如 传了 向 制， 就只  
在 个 向上

### 它是干什么的

取和 大 依 R 个函

## 片段 8：把目标位置翻成方向字符串

函 `_get_direction()`

```
def _get_direction(self, obj_box):
    if not self._player_xywh:
        return None
    dx = obj_box[0] - self._player_xywh[0]
    dy = obj_box[1] - self._player_xywh[1]
    ...
    return "RIGHT_UP" / "LEFT" / "DOWN" ...
```

### 这一段的作用

“ 在 家哪 ” 序 向字 串， 如 `LEFT`

`RIGHT_DOWN`

### 执行逻辑

它会 偏 大小，并 合 个小容差 值，决定 像 上下左右，  
像 向

### 它是干什么的

R 动作层 向 器 后 品 先 R

## 片段 9：真正的移动执行函数

函 `_move()`

```
def _move(self, direction, is_slow=False, _action_cache=None, press_time=0.1,
release_time=0.1) -> str:
    if _action_cache and direction != _action_cache:
        for action in _action_cache.strip().split("_"):
            pydirectinput.keyUp(action.lower())

    for action in direction.strip().split("_"):
        pydirectinput.keyDown(action.lower())

    if not is_slow:
        time.sleep(press_time)
        for action in direction.strip().split("_"):
            pydirectinput.keyUp(action.lower())
```

```
time.sleep(release_time)
for action in direction.strip().split("_"):
    pydirectinput.keyDown(action.lower())

return direction
```

### 这一段的作用

向字符串 变 动作 它 动 为 后 会 到 函

### 执行逻辑

如 向变了, 就先 向, 再 下 向 动带 奏 制,  
动则 平

### 它是干什么的

R 动作层

## 片段 10: 判 是否卡住

函 `_update_motion_state()`

```
def _update_motion_state(self, expect_motion: bool, player_visible: bool) -> bool:
    if not self._player_xywh:
        return False
    if not expect_motion or not player_visible:
        Game._position_history.clear()
        Game._stuck_count = 0
        return False
    ...
    if spread_x < 18 and spread_y < 18:
        Game._stuck_count += 1
    else:
        Game._stuck_count = 0
    return Game._stuck_count >= 2
```

### 这一段的作用

几帧 位 历史 判 不 卡住了

### 执行逻辑

如 序 在 动, 但 几帧 向和 向 小, 就增加卡住  
两 R 个 件, 就 为 卡住

它是干什么的

R 个典型 基于位 围 容差判定

片段 11：卡住恢复

函 \_recover\_from\_stuck()

```
def _recover_from_stuck(self, reason: str) -> None:
    now = time.time()
    if now - Game._last_recover_time < 1.2:
        return
    ...
    pattern = Game._recover_step % 4
    Game._recover_step += 1
    if pattern == 0:
        self._tap_direction(reverse_hint, 0.18)
        self._tap_direction("DOWN", 0.12)
    elif pattern == 1:
        self._tap_direction(reverse_hint, 0.18)
        self._tap_direction("UP", 0.12)
    elif pattern == 2:
        for step in ("RIGHT", "DOWN", "LEFT", "UP"):
            self._tap_direction(step, 0.1)
    else:
        self._tap_direction(reverse_hint, 0.22)
        self._tap_direction(current_hint if isinstance(current_hint, str) else
"RIGHT", 0.1)
```

这一段的作用

判 卡住, 序就会启动 套 困动作

执行逻辑

它不 只做 件事, 四 : 反向后下 反向后上 小  
反向再 向 R 可以 从卡 出

它是干什么的

R 动作层

片段 12：打怪

函 `_kill_monster()`

```
def _kill_monster(self, obj_box) -> None:
    if not self._player_xywh:
        return

    if abs(obj_box[0] - self._player_xywh[0]) < self._attack_x and
abs(obj_box[1] - self._player_xywh[1]) < self._attack_y:
        direction = self._get_direction(obj_box)
        ...
        if face:
            self._key_press(face.lower())
            self._key_press("x")
            self._release_cached_action()
            return

    direction = self._get_direction(obj_box)
    if direction:
        Game._action_cache = self._move(direction, is_slow=True,
_action_cache=Game._action_cache)
```

这一段的作用

处 “去 R 个 ” R 件事 了就 ， 了就

执行逻辑

入 击 围后， 序会先 向 ， 再 `x` 击 入 击 围 ， 就 向 去

它是干什么的

前R 套 不复 ， 就 常 基 型

片段 13：拾取与朝门移动

函 `_pick_up()` 与 `_move_to_door()`

```

def _pick_up(self, obj_box) -> None:
    if not self._player_xywh:
        return
    if abs(obj_box[0] - self._player_xywh[0]) < self._move_x and
abs(obj_box[1] - self._player_xywh[1]) < self._move_y:
        self._key_press("f")
        self._release_cached_action()
        return
    direction = self._get_direction(obj_box)
    if direction:
        Game._action_cache = self._move(direction, is_slow=True,
_action_cache=Game._action_cache)

def _move_to_door(self, obj_box) -> None:
    direction = self._get_direction(obj_box)
    if direction:
        Game._action_cache = self._move(direction,
_action_cache=Game._action_cache)

```

### 这一段的作用

个 东 , 个

### 执行逻辑

品 入 取 围就 f , 否则先 则不 交互 , 只 动即可

### 它是干什么的

R 两 动作层其实 在 复同 套 : 判 否 够 , 则交互, 不 则 动

## 片段 14: 后备路线搜索与向上扫门

函 \_fallback\_up\_search\_move() 与 \_fallback\_route\_move()

```

def _fallback_up_search_move(self) -> None:
    ...
    if phase == 0:
        command = "DOWN"
    elif phase == 1:
        command = "RIGHT"
    elif phase == 2:

```

```

        command = "UP"
    else:
        command = "RIGHT"
    Game._action_cache = self._move(command, is_slow=True,
    _action_cache=Game._action_cache)

def _fallback_route_move(self, direction: str) -> None:
    normalized = direction.upper()
    if normalized == "DOWN":
        ...
    elif normalized == "UP":
        self._fallback_up_search_move()
        return
    ...
    Game._action_cache = self._move(normalized, is_slow=True,
    _action_cache=Game._action_cache)

```

### 这一段的作用

前帧，但存在，序不会完全停住，个受  
后备 动

### 执行逻辑

向上 向，因为多副上不好对准，以会先做个分动作  
向下则会前 向位左下 右下

### 它是干什么的

R " 不 也尽 大 向 " 启发

## 片段 15: 主决策入口开头

函 run() 前半

```

def run(self) -> None:
    raw_player = self._get_cls("player")
    if raw_player is None:
        Game._player_missing_count += 1
        Game._player = Game._pre_player
    else:
        Game._player_missing_count = 0
        Game._player = raw_player
        Game._pre_player = raw_player

```

```
    if raw_player is None and Game._player_missing_count >=
Game._missing_player_recover_threshold:
        self._recover_missing_player()
        return

    if Game._player is None:
        return
```

### 这一段的作用

先 R 帧 否定位到 家 如 , 就临 上 帧位 ; 如  
多帧 不 家, 就 发 复

### 执行逻辑

R 做可以减少 YOLO 偶发 动作 动

### 它是干什么的

R 动作层 入决 前 定位

## 片段 16: 更新玩家中心和卡住检测

函 run() 中

```
self._player_xywh = list(Game._player["xywh"])
self._player_xywh[0] = self._player_xywh[0] + self._player_xywh[2] / 2
self._player_xywh[1] = self._player_xywh[1] + self._player_xywh[3] / 2
logger.info("player center: {}", self._player_xywh[:2])
self._maybe_update_room_entry()

if self._update_motion_state(
    expect_motion=Game._action_cache is not None,
    player_visible=raw_player is not None,
):
    self._recover_from_stuck("movement_stalled")
    return
```

### 这一段的作用

家 中 , 入口 , 并 即 不 卡住

### 执行逻辑



只判为卡住，就会刻入复，后 R 帧不再判

它是干什么的

优先保己动，再V别决

片段 17：战斗优先级

函数 run() 分

```
boss = self._get_class("boss")
if boss:
    nearest_boss = self._get_nearest(boss)
    if nearest_boss:
        self._kill_monster(nearest_boss["xywh"])
        return

monsters = self._get_class("monster")
if monsters:
    nearest_monster = self._get_nearest(monsters)
    if nearest_monster:
        self._kill_monster(nearest_monster["xywh"])
        return
```

这一段的作用

定义优先：先 Boss，后

执行逻辑

处功，前帧即，不会去东

它是干什么的

R 动作层 “单帧只做一件事”

片段 18：拾取优先级

函数 run() 取分

```
goods = self._get_class("goods")
if goods:
    nearest_goods = self._get_nearest(goods)
```

```

    if nearest_goods:
        self._pick_up(nearest_goods["xywh"])
        return

money = self._get_clss("money")
if money:
    nearest_money = self._get_nearest(money)
    if nearest_money:
        self._pick_up(nearest_money["xywh"])
        return

```

### 这一段的作用

可 , 序会 始优先 取 和 币

### 执行逻辑

和 , 只 中了 个 品并 始 取, 前帧就

### 它是干什么的

R 体 出刷图 为 序: 优先于 东 , 东 优先于

## 片段 19: 按路线方向选门并进门

函 `run()` 分

```

doors = self._get_clss("door")
if doors:
    direction_hint = self._current_door_direction()
    ...
    usable_doors = self._filter_backtrack_doors(doors, direction_hint)
    nearest_door = self._get_nearest(usable_doors, direct=direction_hint) if
direction_hint else None
    if nearest_door is None and direction_hint:
        raise LookupError("no door matches route hint")
    if nearest_door is None and usable_doors:
        nearest_door = self._get_nearest(usable_doors)
    if nearest_door:
        self._move_to_door(nearest_door["xywh"])
        return

```

### 这一段的作用

前 完后， 序会 向去 动

执行逻辑

先 向 ，再 回头 ， 后优先 向上 严 匹  
， 会 发后 底 ， 不

它是干什么的

R 实到 前 为 关

片段 20：异常处理和最终兜底移动

函 run() 尾

```
except LookupError:
    pass
except Exception as exc:
    logger.exception("game loop error: {}", exc)
    self._release_cached_action()
    return

fallback_direction = self._current_door_direction()
if fallback_direction:
    if self._should_block_backtrack_direction(fallback_direction):
        self._release_cached_action()
        return
    self._fallback_route_move(fallback_direction)
    return

self._release_cached_action()
```

这一段的作用

前 主 分 功 ， 序会尝 做 个安全 后备动作；如 后备  
向 ， 就停住

执行逻辑

， 序不会 ， 优先做可 底 动， 干  
下 帧

它是干什么的

R 动作 后 安全 , “ 就乱动”

第七章: minimap\_route\_demo.py

这个文件负责什么: R 个单 导 , 不 主刷图入口 它  
“ 图 + + 小地图导 ” R 层 否 常

片段 1: 初始化和截图测试

件主体

```
def main() -> None:
    width = 1067
    height = 600
    detector = Detector("")
    navigator = MiniMapNavigator(map_name="generic")

    while True:
        started = time.time()
        img = pyautogui.screenshot(region=(0, 0, width, height))
        frame_rgb = np.array(img)
        annotated, objects = detector.detect(frame_rgb)
        frame_bgr = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR)
```

这一段的作用

固定区域 图 和导 , 不 入完 刷图主

执行逻辑

初始化 器和导 器后, 取屏幕 定区域, 并 和导 块

它是干什么的

R 个 便 发 导

片段 2: 打印导航结果并显示调试窗口

件后半

```
snapshot = navigator.build_route_snapshot(frame_bgr, objects or [])
print(
    json.dumps(
        {
            "current_room": snapshot.current_room,
            "boss_room": snapshot.boss_room,
            "query_room": snapshot.query_room,
            "elite_room": snapshot.elite_room,
            "next_room_direction": snapshot.next_room_direction,
            "selected_door_center": snapshot.selected_door_center,
            "cost_ms": round((time.time() - started) * 1000, 1),
        },
        ensure_ascii=False,
    )
)
```

这一段的作用

导 印 JSON, 同 大图 和小地图 口

执行逻辑

R 发 可以 常 到 “导 层 刻 为 己在哪 在哪 下 哪

它是干什么的

R 个专 导 否准 体 工具

A

附录：为什么 olaplug 不放进主线逐段讲解

不是遗漏，而是为了抓主线。 olaplug 主 个功 常大 DLL 封 层， 口 定 入 图 OCR YOLO 内存 口 大 函 但 前R 套刷图主 并 实例化 OLAPlugServer , 也 BindWindow()

当前真实执行链

你 在 住 主

win32gui 找窗口

- > pyautogui 截图
- > detector.py 做 YOLO 检测
- > minimap\_nav.py 做小地图识别并调用 A 星
- > game.py 用 pydirectinput 发方向键 / 攻击键 / 拾取键

### 这一段的作用

你 力优先 在 前 刷图主 上

### 执行逻辑

如 你 始就 olaplug , 容 大 口 , 反 不 R 套刷图  
代 在到底 么工作

### 它是干什么的

帮助你 主 , 不 备 工具 带偏

基于 前工作区 D:\yolo\yolo11\dnf , 修复了 中 乱 和 多